


# Universidad Nacional Autónoma de México

Facultad de Ingeniería

Computer Engineering

## Compilers (Group 5)

E.M. René Adrián Dávila Pérez

# PROJECT REPORT: SYNTAX ANALYZER (PARSER) & SYNTAX DIRECTED TRANSLATION (SDT)

architecture package: `unam.fi.compilers.g5.05`

**Team members 05:**

319088058 *(Documentation)*
321634421 *(Project Manager)*
321037530 *(Software developer)*
424490412 *(State Machine developer)*
321009298 *(Presenter)*

**Group:** 5
**Semester:** 2027-1

Ciudad Universitaria, CDMX. November 2026
Repository: https://github.com/alanvilchisssss/Compilers_Repository

# Contents

**1 Introduction** **2**
- 1.1 Problem approach . . . 2
- 1.2 Motivation . . . 2
- 1.3 Objectives . . . 2

**2 Theoretical Framework** **3**
- 2.1 From Lexical Analysis to Predictive Top-Down Parsing . . . 3
- 2.2 Left-Factored Context-Free Grammar (CFG) . . . 3
- 2.3 Syntax-Directed Translation (SDT) and Attributes . . . 4
- 2.4 Formal Table of SDT Semantic Rules . . . 5

**3 Development** **5**
- 3.1 State Machine Design of the $LL(1)$ Parser . . . 5
- 3.2 Functional Description of the Implementation . . . 8
- 3.3 Description of the Executed Tests . . . 8

**4 Results** **9**

**5 Conclusions** **12**

# 1 Introduction

## 1.1 Problem approach

In this first phase of this project, we built a syntax analyzer, which is capable of reading code in the console of the program, or in a .txt file, which the programs reads, and based of the content classifies the lexemes in 5 different categories: reserved words, identifiers, operators, constants and punctuation signs. It's important to mention that our lexer only verifies that each word exists as an isolated form inside of the vocabulary that we defined previously in the project. Therefore, we don't validate the order, or structural order in which the words appear in the program or .txt file. In this second phase of our project, we are going to design an implement de Syntax Analyzer, and the Syntax-Directed Translation, which inside of the package code of unam.fi.compilers.g5.05, applying only the theoretical concepts seen in class, without any other automatic generator or libraries. The system revives the sequence of tokens generated by the lexer, builds a representation of the parsing three by using a predictive model known as Top Down LL(1), modeled on it's own state machine and afterwards verifies that the text actually follows the semantic rules of the SDT, distinguishing between syntactical errors, and semantic errors.

## 1.2 Motivation

The syntax analysis is a fundamental part in the theory of formal languages and the design of compilers because of: [1]:

- **Hierarchic validation:** A Context free grammar defines the hierarchy of sentences (as declarations and variables) allowing that a predictive parser verifies the grammar in an ideal time without extra process that might be inefficient for a huge load of data, even though, we aren't processing a huge quantity of data, it could mean time and memory for the program.
- **Verifications depending on the context:** Because of a CFG on its own cant impose rules sensible to the context, like proving that a variable doesn't receive a value of another type which doesn't belong, or verifying previous declarations, implementing a SDT in the parsing three gives one mechanism to manage the attributes and validations in the .txt.
- **Precise mistake manager:** Distinguishing between structural mistakes of the grammar (`Parsing error...`) and the violations of the type or context of the program(`Parsing Success!`, followed by `SDT error...`) can give the user the right instructions in why is our program right or wrong.

## 1.3 Objectives

**General Objectives:**
The general objective of this part of the project is building a syntax analyzer as a Top-Down strategy, using a verifier of SDTs in out program `unam.fi.compilers.g5.05`,

capable of validating sentences based of the tokens of the lexer, and using the different results of analyzing the semantic.

**Specific Objectives:**

- Adapting the output of the lexer analyzer so that we can manage the flow of the tokens so that the lexeme is conserved on it's own lexer category, line number and marker of the end of the input $.
- Modeling as a state machine and implementing a predictive parser descendant ($LL(1)$) based in the CFG, factorized by the left in the first release of the project.
- Defining an implementing the semantic rules SDT through synthesized attributes and the heritage for the verification of the compatibility in types and declarations and initialization, (as `int x = 10;`), the calling format to `print` / `printf` and the register of a symbol table.
- Demonstrate theoretical and in the implementation the three scenarios required: Syntax and semantical verification, syntax success and semantical mistake, and syntactical mistakes.

# 2 Theoretical Framework

## 2.1 From Lexical Analysis to Predictive Top-Down Parsing

According to the classic theory of compiler design [1], the syntax analyzer receives a string of tokens coming from the lexical analyzer and checks if this string can be generated by the Context-Free Grammar (CFG) of the source language.

A **Top-Down Parser** builds the parse tree from the root (the start symbol) down to the leaves (the terminal tokens), trying to find a leftmost derivation for the input string. In particular, a **predictive LL(1) parser** scans the input from left to right (first $L$), produces a leftmost derivation (second $L$), and selects the exact production to apply by looking at only 1 lookahead token without requiring backtracking.

## 2.2 Left-Factored Context-Free Grammar (CFG)

To guarantee a deterministic $LL(1)$ behavior without ambiguities caused by common prefixes, we use the left-factored Context-Free Grammar $G = (V, \Sigma, R, \text{Program})$, extended with the end-of-input marker $ and support for multiple statements:

$$\text{Program} \rightarrow \text{StmtList \$} \tag{1}$$
$$\text{StmtList} \rightarrow S\ \text{StmtList} \mid \varepsilon \tag{2}$$
$$S \rightarrow \text{Decl} \mid \text{Call} \tag{3}$$
$$\text{Decl} \rightarrow \text{type } \textbf{id}\ \text{Decl'} \tag{4}$$
$$\text{Decl'} \rightarrow ; \Big| = \text{Expr} ; \tag{5}$$
$$\text{Call} \rightarrow \text{print_kw} (\ \text{Expr Call'} \tag{6}$$
$$\text{Call'} \rightarrow ) ; \Big| , \text{Arg} ) ; \tag{7}$$

where the lexical non-terminals derive into the terminal categories provided by the lexer:

$$\text{type} \rightarrow \textbf{int} \mid \textbf{float} \mid \textbf{char} \mid \textbf{void} \tag{8}$$
$$\text{print_kw} \rightarrow \textbf{printf} \mid \textbf{print} \tag{9}$$
$$\text{Expr} \rightarrow \textbf{id} \mid \textbf{constant} \tag{10}$$
$$\text{Arg} \rightarrow \textbf{id} \mid \textbf{constant} \tag{11}$$

By factoring the common prefixes ($\alpha_1$ = type **id** in declarations and $\alpha_2$ = print_kw ( Expr in print calls), the parser consumes the shared prefix first and uses the single lookahead token (; vs. = in Decl', or ) vs. , in Call') to choose the valid production in a deterministic way.

## 2.3 Syntax-Directed Translation (SDT) and Attributes

A Context-Free Grammar on its own cannot verify context-sensitive rules, such as checking if the value assigned to a variable matches its declared type or if an identifier was already declared before [1]. To solve this, we use a **Syntax-Directed Definition (SDD)** implemented through a **Syntax-Directed Translation (SDT)** scheme.

An SDT scheme associates attributes with each node of the parse tree and evaluates semantic rules over the constructed tree:

- **Synthesized attributes:** They are calculated in a parent node from the values of its child nodes (information goes up from the leaves to the root). For example, in the production Expr → **constant**, the node Expr synthesizes its attribute Expr.type by checking if the lexeme of the constant is an integer (`int`), a float (`float`), or a string (`string`).
- **Inherited attributes:** They are passed from a parent node or a left sibling to a child node (information goes down or moves sideways to the right). In the factored rule Decl → type **id** Decl', the type declared in the left child (type.type) is passed as an inherited attribute (Decl'.inh_type = type.type) to the sibling node Decl' so that, when Decl' evaluates an assignment (= Expr ;), it knows which data type it must compare Expr.type against.

### 2.4 Formal Table of SDT Semantic Rules

Table 1 shows the formal design of our Context-Free Grammar along with the SDT semantic actions associated with each production, where SymTab represents the Symbol Table.

## 3 Development

Following the delivery instructions, the system was structured under the architecture package `unam.fi.compilers.g5.05` without depending on external tools for syntax generation.

### 3.1 State Machine Design of the $LL(1)$ Parser

To model the deterministic behavior of the syntax analyzer from the left-factored grammar, the finite state machine shown in Figure 1 was built using TikZ vector code.

The automaton starts from the **Start** state and checks the lookahead token to branch without ambiguity into two main paths:

1. **Left branch (Declarations — Decl):**
   When receiving a type reserved word (`int`, `float`, `char`, `void`), it moves to the *Type read* state, and after consuming an `identifier`, it advances to the *Id read* state. At this exact point, the left factoring (Decl') takes place:
   - If the next token is the delimiter `;`, it goes directly to the final **Accepted** state (simple declaration).
   - If the next token is the operator `=`, it moves to the *Assignment* state, requires an `identifier` or `constant` to reach *Expr read*, and finally consumes `;` to reach the **Accepted** state.
2. **Right branch (Output calls — Call):**
   When reading `print` or `printf`, it advances to *Function read*, requires the delimiter `(` towards *Open parenthesis*, and consumes a `constant` or `identifier` to place itself in *Arg 1 read*. In this state, the factoring of Call' works as follows:
   - If it receives `)`, it goes to *Close parenthesis* and with `;` reaches the **Accepted** state.
   - If it receives a comma (`,`), it moves to *Comma read*, consumes the second argument (*Arg 2 read*), closes with `)` towards *Close parenthesis*, and finishes with `;` in **Accepted**.

From the **Accepted** state, if there are more tokens in the input flow, the automaton takes the *next statement* transition back to **Start** (representing the recursion of StmtList), or it stays there and finishes successfully when finding the end-of-file marker `$(EOF)`. Any token outside the described transitions causes an immediate rejection and outputs `Parsing error...` on the corresponding line.

Table 1: Productions of the Context-Free Grammar and associated SDT semantic rules.

| **CFG Production** | **Semantic Rule / SDT Verification Action** |
|---|---|
| Decl → type **id** Decl' | 1. Verify that type.type ≠ `void` and **id**.lexeme ∉ SymTab.<br>2. Pass inherited attribute: Decl'.inh_type ← type.type.<br>3. If Decl'.ok == True, register SymTab[**id**.lexeme] ← type.type; otherwise emit `SDT error...`. |
| Decl' → ; | Simple declaration without initialization: Decl'.ok ← True. |
| Decl' → = Expr ; | Check type compatibility between the inherited and synthesized attributes:<br>If Decl'.inh_type == Expr.type, then Decl'.ok ← True; otherwise emit `SDT error...` on the current line. |
| Call → print_kw ( Expr Call' | Verify that the first argument of `print` or `printf` is a valid string literal:<br>If Expr.type == `string` and Call'.ok == True, it is valid; if not, emit `SDT error...`. |
| Call' → ) ; | Simple call without extra arguments: Call'.ok ← True. |
| Call' → , Arg ) ; | If Arg.type ≠ `error`, then Call'.ok ← True; if not, emit `SDT error...`. |
| type → **int** \| **float** \| **char** \| **void** | Synthesized attribute: type.type ← lexeme of the recognized type. |
| Expr → **constant** | Synthesized attribute Expr.type ∈ {`int`, `float`, `string`} according to the format of the literal. |
| Expr → **id** | Search **id**.lexeme in SymTab. If it exists, Expr.type ← SymTab[**id**.lexeme]; if it was not declared before, emit `SDT error...`. |

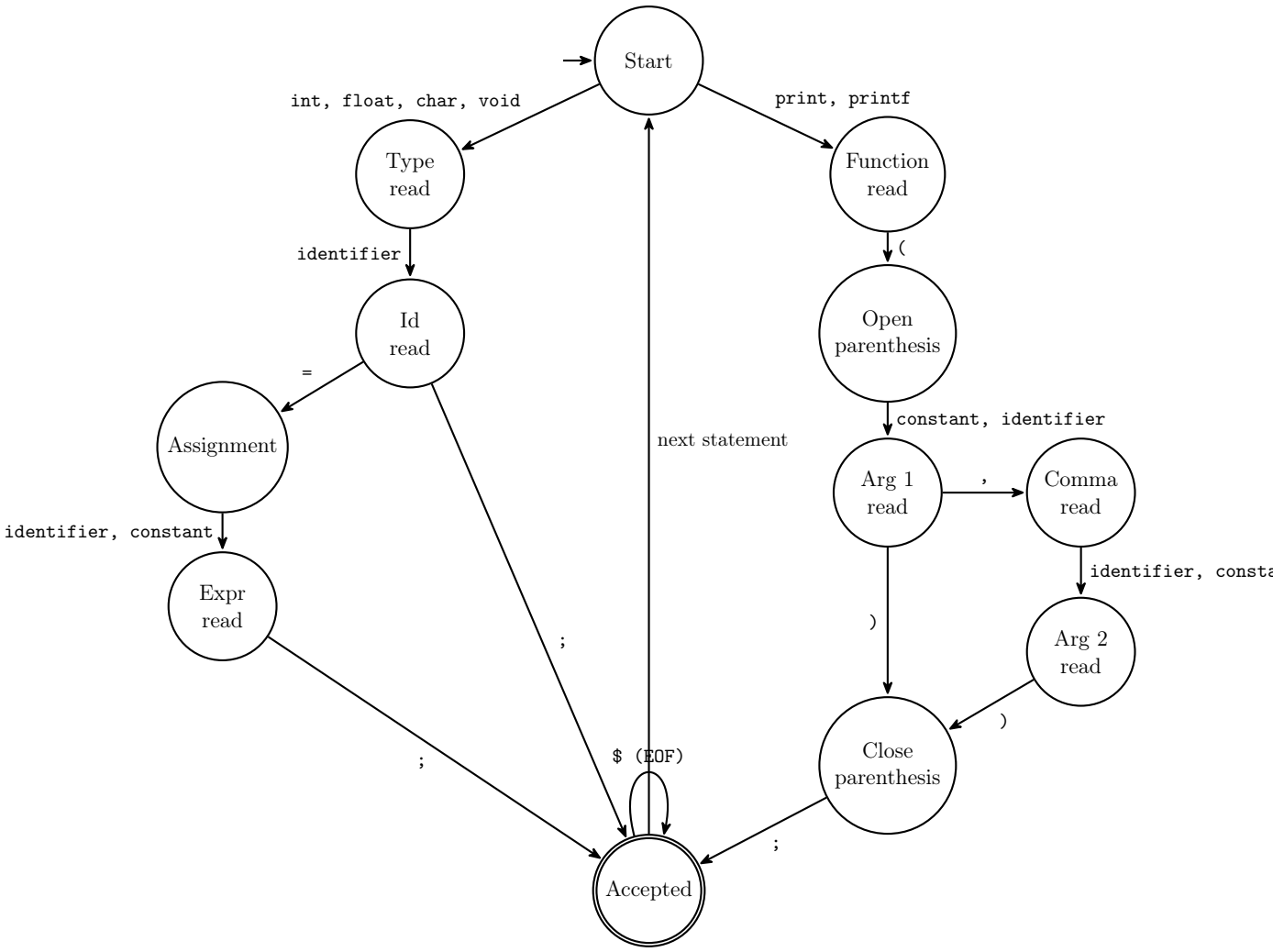



Figure 1: State machine of the predictive $LL(1)$ parser for the syntax analyzer.

## 3.2 Functional Description of the Implementation

Following the methodological guidelines, below we conceptually describe the functional architecture implemented in the four sequential modules of the system:

1. **Input Acquisition Module (`cadena_input`):**
   It keeps the dual interface that allows the user to enter a statement directly from the interactive terminal or read a source code file with a `.txt` extension using exception handling.

2. **Lexical Analysis Module with Line Tracking (`ANALIZADOR_LEXICO`):**
   It splits the input flow line by line and applies the master regular expression pattern that removes single-line and multi-line comments. Unlike the first phase, besides grouping lexemes by category, it builds an ordered sequential list of dictionaries where each token stores three attributes: its category (`tipo`), its original string (`lexema`), and its row number (`linea`), attaching the sentinel token `EOF` ($) at the end.

3. **Syntax Analysis Module (`PARSING`):**
   It directly implements in code the state machine from Figure 1 using a current position index and a helper consumption operation (`consumir`). It sequentially validates the factored productions Decl and Call. If every transition is satisfied until reaching `EOF`, it builds the node structure of the parse tree (`arbol_sintactico`) and outputs `Parsing Success!`; if an invalid transition happens, it stops the process and returns the exact line of the syntax error.

4. **SDT Semantic Verification Module (`verificar_sdt`):**
   It receives the parse tree generated by the parser and initializes a Symbol Table structure (`tabla_simbolos`). It goes through each validated statement applying the rules from Table 1: it checks that no variables are declared as `void`, that there are no duplicated identifiers or variables used without a previous declaration, that in every assignment the inherited attribute of the variable type matches the synthesized attribute of the expression (`obtener_tipo_constante`), and that the first argument of `print`/`printf` is strictly of type `string`.

## 3.3 Description of the Executed Tests

According to the project specifications, three representative test scenarios were designed and theoretically justified:

1. **Case 1 — Valid Syntax and Verified SDT Semantics (`int x = 10;`):**
   **Theoretical argument:** The lexer produces the sequence [`keyword(int)`, `identifier(x)`, `opera`
   The $LL(1)$ parser follows the state machine path *Start* → *Type read* → *Id read* → *Assignment* → *Expr read* → *Accepted*, printing `Parsing Success!`. Afterwards, the SDT verifier propagates the inherited attribute Decl'.inh_type = `int` and compares it against the synthesized attribute Expr.type = `int` deduced from the integer constant `10`. Since the equality `int` == `int` is met and there

is no previous declaration of `x` in the Symbol Table, the variable is registered and the system outputs `SDT Verified!`.

2. **Case 2 — Valid Syntax but SDT Semantic Error (`int x = "hello";`):** **Theoretical argument:** At a context-free level, both the number `10` and the string `"hello"` belong to the terminal category `constant`. Because of this, the sequence [`keyword(int)`, `identifier(x)`, `operator(=)`, `constant("hello")`, `punctuation(;)`] satisfies all the transitions of the parser state machine and produces `Parsing Success!`. However, during the SDT semantic evaluation over the built tree, the inherited attribute requires an integer type (Decl'.inh_type = `int`), while the literal in quotes synthesizes a string type (Expr.type = `string`). The compatibility rule `int` == `string` is false, triggering `SDT error...` and reporting `error in line 1`.

3. **Case 3 — Direct Syntax Error (`int x = 10` without a semicolon):** **Theoretical argument:** When evaluating the assignment branch in the state machine, the parser consumes `=` and the constant `10`, reaching the *Expr read* state, where the only valid transition towards *Accepted* requires the terminal delimiter `;`. Instead, the analyzer finds the end-of-input marker $ (`EOF`). Since there is no transition for $ from *Expr read*, the derivation is rejected before completing the tree and the system outputs `Parsing error...`. Given that the SDT scheme needs a valid parse tree to evaluate its attributes, the syntax failure prevents the semantic verification and consequently reports `SDT error...` and `error in line 1`.

# 4 Results

Below we present the results obtained when running the system under the different evaluation cases.

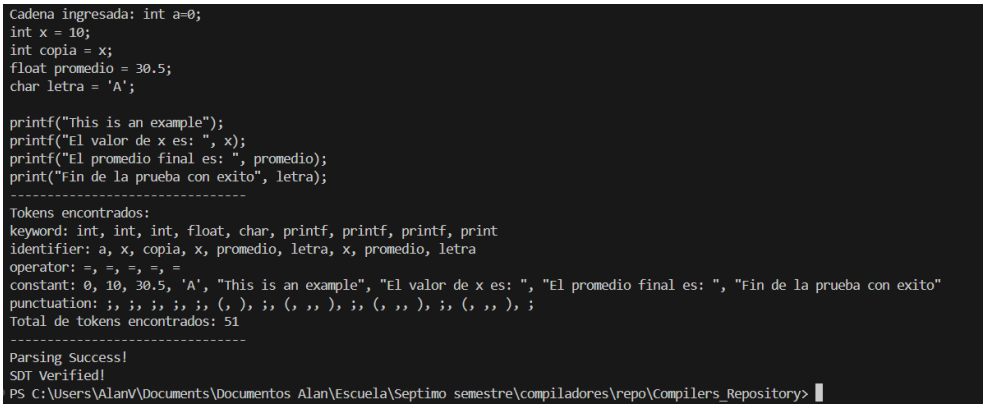



Figure 2: Successful execution in both the $LL(1)$ syntax analysis and the SDT semantic verification for the statement `int x = 10;`.

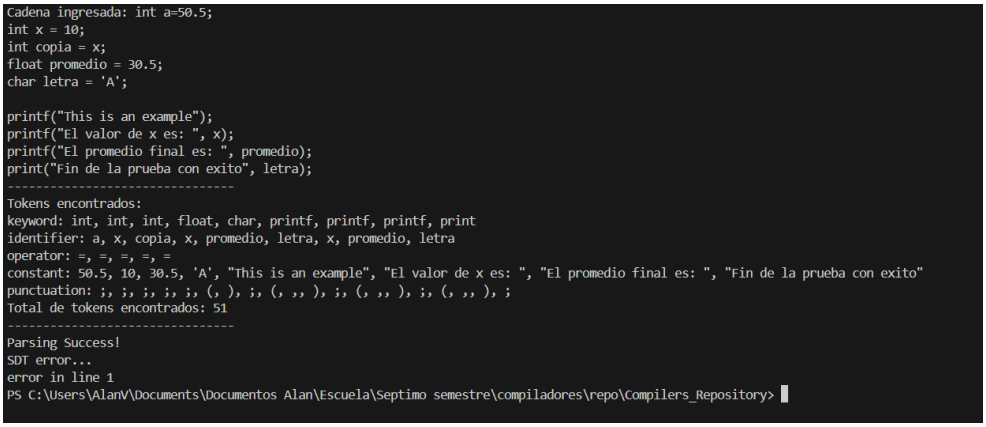



Figure 3: Execution with a valid syntactic structure but SDT semantic rejection due to type incompatibility.

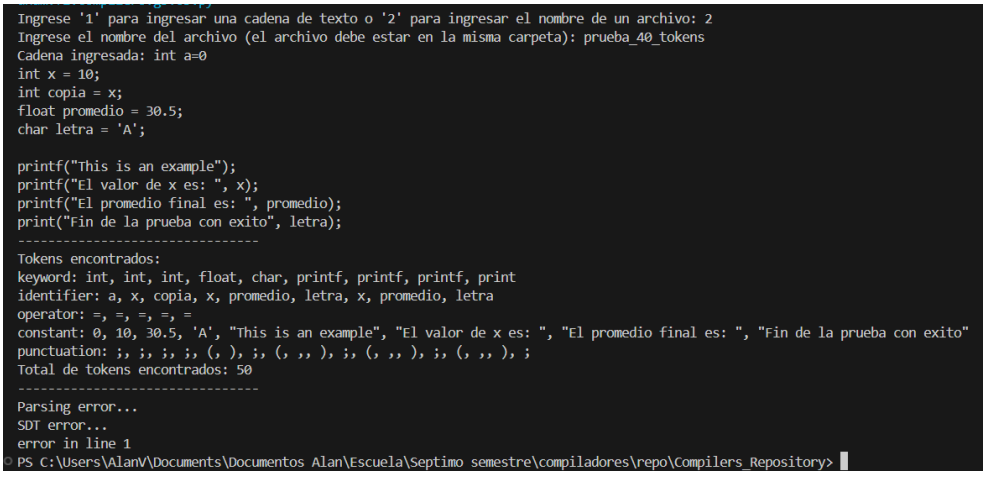



Figure 4: Detection of a direct syntax error when omitting the end-of-statement delimiter.

Table 2: Comparative summary of the cases evaluated in the Parser and SDT verifier.

| **Test Input** | **Parser Output** | **SDT Output** | **Technical Justification** |
|---|---|---|---|
| `int x = 10;` | `Parsing Success!` | `SDT Verified!` | Satisfies the Decl rule and attributes match (`int` == `int`). |
| `printf("Example");` | `Parsing Success!` | `SDT Verified!` | Satisfies the factored Call rule and the first argument is a `string`. |
| `int x = "hello";` | `Parsing Success!` | `SDT error...` | Valid syntax, but SDT fails due to type mismatch (`int` $\neq$ `string`). |
| `int x = 30.5;` | `Parsing Success!` | `SDT error...` | Valid syntax, but SDT fails due to type incompatibility (`int` $\neq$ `float`). |
| `void a = 10;` | `Parsing Success!` | `SDT error...` | Grammatically `void` is a `type`, but SDT forbids `void` variables. |
| `int x = 10` | `Parsing error...` | `SDT error...` | Missing the required terminal `;` at the end of the declaration. |
| `int = 10;` | `Parsing error...` | `SDT error...` | Missing the required `identifier` token after the `int` type. |

# 5 Conclusions

The theoretical and experimental analysis of this second phase allows us to establish the following technical conclusions:

- **Direct correspondence between Left Factoring and the $LL(1)$ State Machine:** The factoring applied in the first delivery allowed us to model the syntax analyzer through a deterministic automaton free of ambiguous transitions. By delaying the branching until the states *Id read* (to choose between `;` and `=`) and *Arg 1 read* (to choose between `)` and `,`), the parser works in linear time $O(n)$ by checking a single lookahead token without requiring backtracking.
- **Separation between context-free grammars and context-sensitive rules (SDT):** The modular architecture proved that syntactic validity is a necessary condition, but it is not enough to guarantee the coherence of a program. Cases like `int x = "hello";` prove that the parser only validates the structural sequence of lexical categories, while the SDT scheme has the responsibility of propagating inherited and synthesized attributes along with the Symbol Table to detect type mismatches.
- **Propagation of lexical metadata for accurate diagnosis:** Enriching the structure of each token from the lexical analyzer with its source line number and adding the end-of-file sentinel symbol (\$) was essential to synchronize the acceptance transitions of the automaton and output exact diagnostics under the `error in line n` standard.

# References

[1] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd. USA: Addison-Wesley Longman Publishing Co., Inc., 2006. ISBN: 0321486811.